Lab 06 — Decision Trees and k-Nearest Neighbours, Built from Scratch

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 06 · Topic 5. Lecture: `slides/06_trees_knn.pdf`. Demo notebooks in `notebooks/06-trees-knn/`: `01_decision_trees.ipynb` and `02_knn.ipynb`.

Notebook for this lab: `notebooks/06-trees-knn/lab06_trees_knn.ipynb`, with helpers in `lab06_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the plots and prints the comparisons once your functions work.

Solution: `solutions/lab06_trees_knn_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first.

The rule of this lab. Trees and KNN have no gradient to descend: they are *algorithms*. You write the tree with NumPy and the distances of KNN with `jax.numpy`, and compare each with its scikit-learn twin on the **same split and the same scaling**. The notebook prints the difference; explain it (ties, `float32`, a tolerance).

In class (60 minutes): Parts A, B and C, the tree from scratch. **At home: Parts D to H**, the depth and pruning, the regression tree, KNN and the challenge; the solution notebook explains them.

By the end of the lab you should be able to:

1. Compute the Gini index and the entropy of a node and the gain of a question, and find the best threshold of a numeric feature with one sorted scan.

2. Grow a classification tree recursively and read it as rules; choose its depth on validation data and prune it.
3. Grow a regression tree (variance reduction) and see that it predicts a staircase and cannot extrapolate.
4. Write KNN with broadcasting and `top_k`, and show the effect of scaling, of k and of irrelevant features.
5. Compare a tuned tree and a tuned KNN with a paired test over repeated splits.

Part	Task	twin	Time
A	Data visualization and evaluation methodology: breast cancer, German credit, concrete	—	10 min
B	Impurity and the best split: Gini, entropy, one sorted scan	`DecisionTreeClassifier(max_depth=1)`	20 min
C	A classification tree from scratch: recursion, prediction, rules	`DecisionTreeClassifier`	30 min
D	Overfitting: the depth on validation data, pruning	`cost_complexity_pruning_path`	home
E	A regression tree: variance reduction	`DecisionTreeRegressor`	home
F	KNN with `jax.numpy`: distances, neighbours, vote; scaling, k, noise	`KNeighborsClassifier`	home
G	KNN for regression	`KNeighborsRegressor`	home

Part	Task	twin	Time
H	Challenge: the symbolist and the analogizer on repeated splits	`GridSearchCV`, `LogisticRegression`	home

Setup

```
cd faa
source venv/bin/activate        # see README: make venv, or pip install .
jupyter lab notebooks/06-trees-knn/lab06_trees_knn.ipynb
```

The data are in `datasets/` (read with polars, see `datasets/README.md`): **breast cancer** (569 tumours, 30 measurements), **German credit** (1000 applications, 61 columns after the one-hot encoding of the text attributes, 30% bad credits), **concrete** (1030 mixtures, the strength in MPa) and the **digits**. `stratified_split`, `fit_standardizer` and `standardize` are your Class 03 functions, given ready-made; the trees are plain dictionaries and `print_tree` writes one as rules.

Part A — Data Visualization and Evaluation Methodology

The notebook loads the three data sets, makes the stratified split, and prints the baselines. It draws one feature of each classification problem by class and the concrete strength against the age.

1. What are the accuracies of the "always the majority class" baseline on breast cancer and German credit, and the RMSE of the "always the mean" baseline on concrete? What must a model beat?
2. The histogram of `worst_perimeter` by class and the scatter of the strength against the age: what kind of model does each suggest?

Part B — Impurity and the Best Split

A node holds class counts $n_1, \dots, n_K$ ($p_k = n_k/n$). Its **Gini index** is $G = 1 - \sum_k p_k^2$ and its **entropy** $H = -\sum_k p_k \log_2 p_k$. The **gain** of the question

"$x \leq t$" is the impurity of the node minus the weighted impurities of the two children.

B1. The criteria and the best threshold

Implement `gini(counts)` and `entropy(counts)` for arrays whose **last axis** holds the class counts (any leading axes), and `best_threshold(x, y, n_classes, criterion)`: sort `x`, take the cumulative class counts with `np.cumsum` of the one-hot labels, evaluate the gain of **every** split between two different consecutive values at once, and return `(threshold, gain)` with the threshold at the midpoint (`(None, 0.0)` if no split has a positive gain). The driver prints the gains of the 14-day "play tennis?" table of the lecture and compares your best question on breast cancer and German credit with the root of `DecisionTreeClassifier(max_depth=1)`.

1. Which question is the root of the tennis tree, under each criterion? What are the entropy and the Gini index of the parent?
2. Do your feature and threshold agree with scikit-learn's? Which is the best question on each data set, in words?
3. What is the cost of `best_threshold` for n examples, and what would it be without the cumulative counts?

Part C — A Tree from Scratch

C1. Grow it and use it

Implement `fit_tree(X, y, n_classes, max_depth, min_leaf, criterion, depth=0)` returning a dictionary node (`counts`, `pred`, `n`, and for internal nodes `feature`, `threshold`, `left`, `right`): a **leaf** when the node is pure, the depth limit is reached, or it has fewer than `2 * min_leaf` examples; otherwise the best question over all features, rejecting splits that leave fewer than `min_leaf` examples on a side, and the recursion on the two parts. Implement `predict_tree(node, X)`. The driver compares your tree with `DecisionTreeClassifier` at depths 2, 4 and 6 and prints your depth-3 tree for breast cancer as rules.

1. How close is your tree to scikit-learn's (leaves, predictions)? Where do the differences come from?
2. Read the printed rules: which question comes first, and why is a leaf with a single example suspicious?

3. How do the accuracies change with the depth on each data set?

Part D — Overfitting and the Depth

D1. The depth on validation data

Implement `best_depth(Xtr, ytr, Xva, yva, n_classes, depths, min_leaf)`: fit one tree per depth, measure the accuracy on the training and validation parts, and return the depth with the best validation accuracy (the smallest on a tie) and the two curves. The driver draws them and compares the best depth with the unlimited tree, and then with **cost-complexity pruning** chosen by cross-validation.

1. Which depth is best on each data set? How does the unlimited tree do on the test part, and how many leaves does it have?
2. How many leaves does the pruned tree keep, and what is its test accuracy? What does the German-credit result say about the data?

Part E — A Regression Tree

E1. Variance reduction

Implement `best_threshold_reg(x, y)` (the same sorted scan with cumulative sums of y and y^2; SSE $= \sum y^2 - (\sum y)^2/n$ for each side) and `fit_reg_tree(X, y, max_depth, min_leaf)` (leaf value = the mean). The driver compares your tree with `DecisionTreeRegressor` on the concrete data at four depths, with a linear regression, and plots the predictions against the actual values.

1. How close are the two trees, and why do they differ at large depths?
2. Which model is best on concrete, and why? What does the predicted-against-actual plot of a tree look like, and why?

Part F — KNN with `jax.numpy`

F1. Distances, neighbours, vote

Implement `pairwise_sq_dists(A, B)` (shape `(len(A), len(B))`, with broadcasting: $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b$) and `knn_predict(Xtr, ytr, Xte, k, n_classes, weights)`: the k nearest training points (`jax.lax.top_k`

of the negative squared distances) vote, the sum of their one-hot labels, with `weights="distance"` each vote multiplied by $1/(d + 10^{-9})$. The driver compares with `KNeighborsClassifier` for $k = 1, 5, 15$, then shows raw against standardized features, the curve of k on the moons and the effect of added noise features against a tree and logistic regression.

1. How close are your distances and predictions to scikit-learn's? Why is there no randomness to explain a difference?
2. What does standardization change on breast cancer? What happens to the training and test accuracy as k grows from 1 to 201?
3. What happens to KNN, to the tree and to logistic regression when 1000 noise columns are added? Why?

Part G — KNN for Regression

G1. The local mean

Implement `knn_regress(Xtr, ytr, Xte, k)`: the mean target of the k nearest training points. The driver compares with `KNeighborsRegressor` on the concrete data for several k, and with the linear model and your tree.

1. Which k is best? What happens at $k = 1$ and at $k = 50$?
2. Order the linear model, the tree and KNN on concrete and explain the order.

Part H — Challenge: the Symbolist and the Analogizer

Choose a data set of Project 1 (the notebook uses German credit). Compare a **decision tree** (`min_samples_leaf` chosen by 5-fold cross-validation), a **KNN** (the number of neighbours chosen by cross-validation) and **logistic regression** on repeated stratified splits.

H1. Repeated splits

Implement `repeated_scores(make_model, X, y, n_rep, test_frac)`: for each repetition a stratified split with `np.random.default_rng(rep)`, standardization with the **training** statistics, the model from `make_model()`, and the array of test accuracies.

1. Which learner has the best mean accuracy and which the largest spread? Which differences does the corrected paired t-test of Class 05 find significant?
2. Repeat on another data set: breast cancer, `wine_quality` or `heart_disease` (prepare it with polars). Does the order change? What would you recommend for your project?

Going further. Add the SVM of Class 05 and a Keras MLP to the comparison, and put the cost (training and prediction time) next to the accuracy. Or show on a two-dimensional problem the decision boundary of your tree and of your KNN side by side (`plot_boundary`).